LAPORAN

“Studi Kasus Bioskop 212”



Nama : Kristian Utama Putra
NRP : 3125600047
Dosen : Grezio Arifiyan Primajaya , S.Kom, M.Kom

PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TAHUN 2026

TUGAS

1. Rancang dan implementasikan menggunakan pendekatan OOP berdasarkan

kebutuhan bisnis

2. Berdasarkan kelas yang sudah ditentukan, silahkan tentukan : - atribut; - tipe data; - access modifier; - constructor; - method; - parameter method; - return value; - hubungan antar kelas.

3. Buat UML Class Diagram berdasarkan desain yang kalian buat.

4. Implementasikan seluruh desain tersebut ke dalam program Java.

5. Setiap class harus memiliki interaksi dengan kelas yang lain, sesuai dengan

kebutuhan bisnis.

6. Program yang dikembangkan harus memiliki minimal 1 alur proses bisnis utama

sesuai dengan studi kasus

7. Wajib menerapkan : - Enkapsulasi - Polymorphism - array/array of object untuk menyimpan dan mengolah beberapa object.

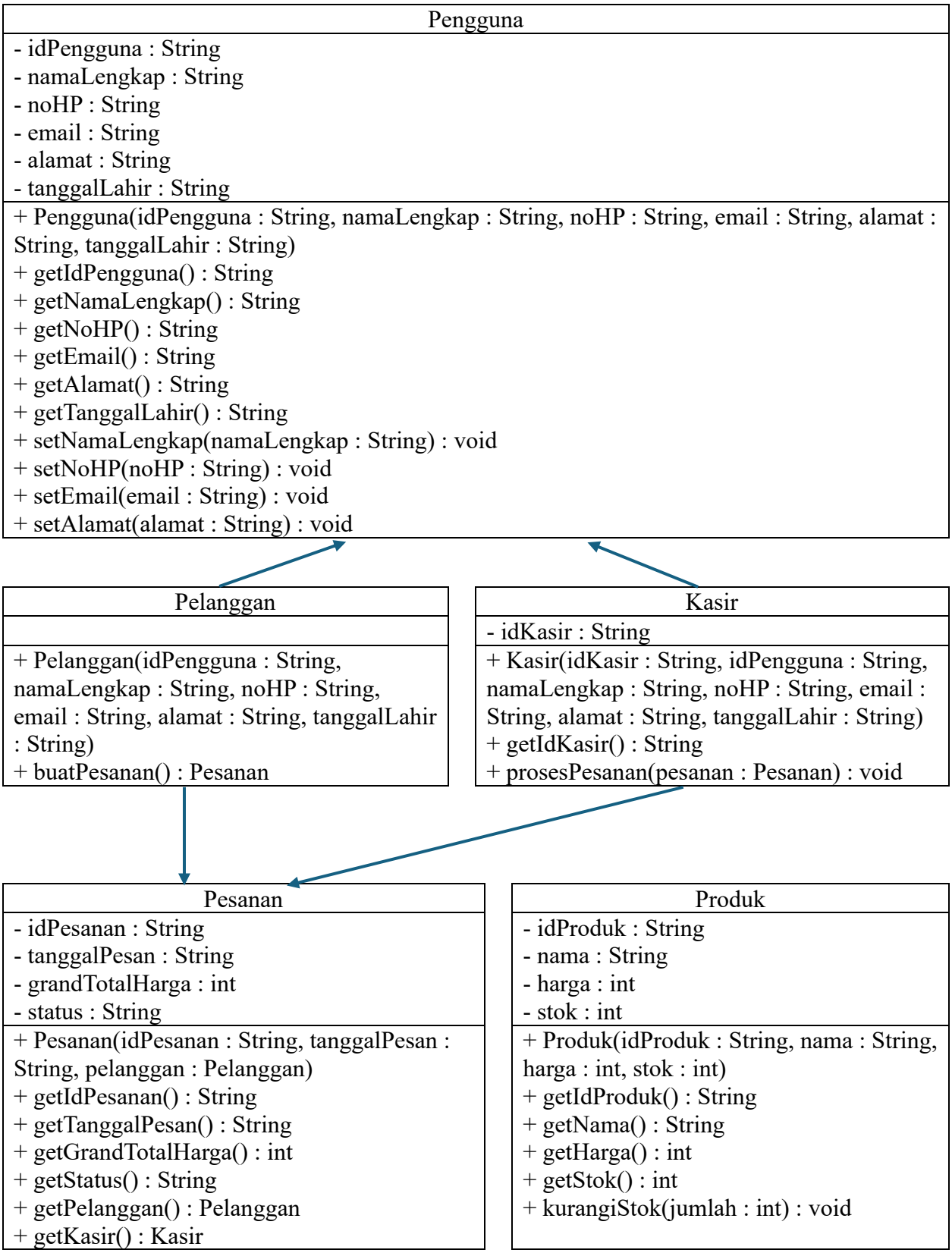
8. Pastikan program yang dikembangkan memiliki main()

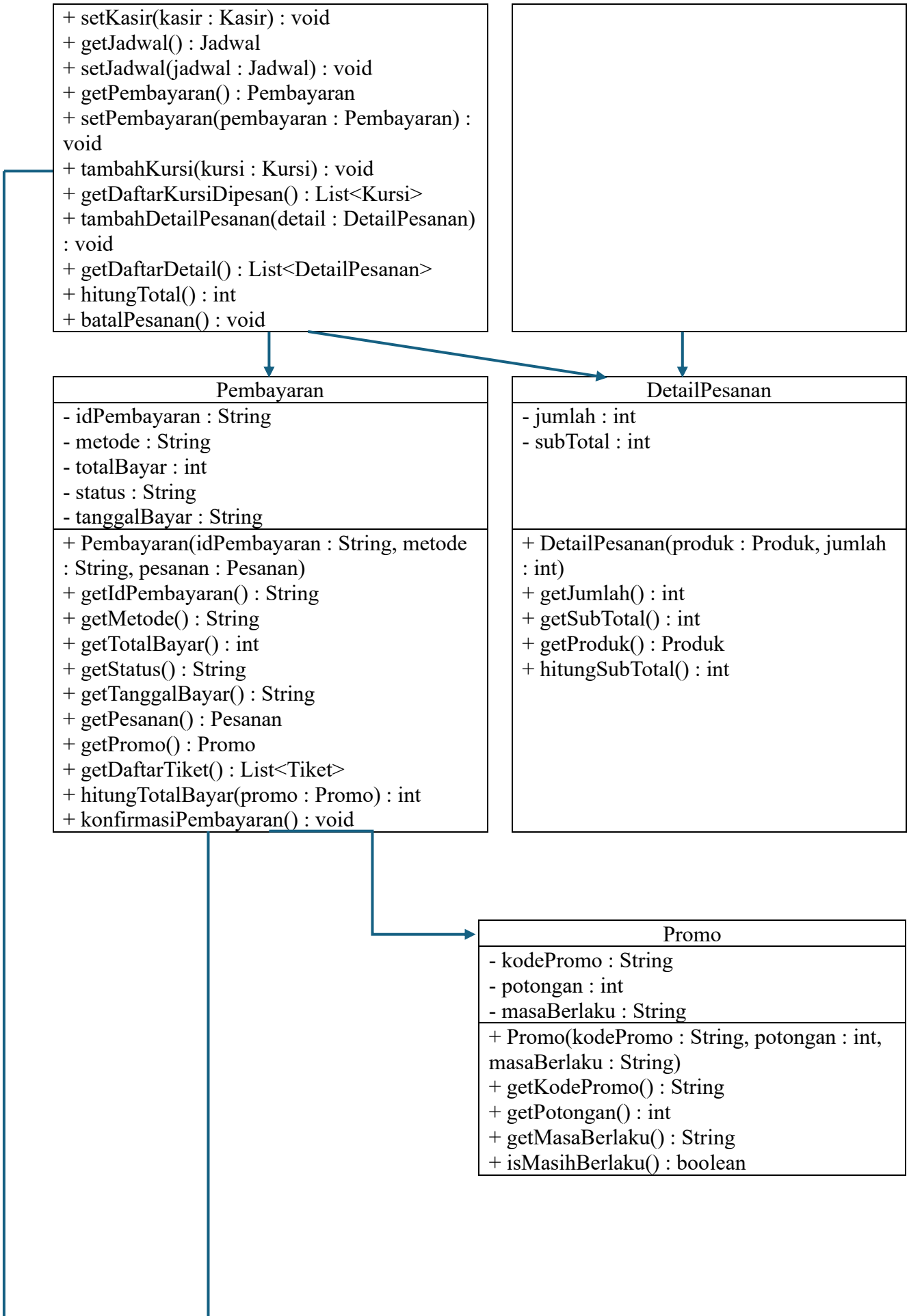
9. Program harus dapat dikompilasi dan dijalankan.

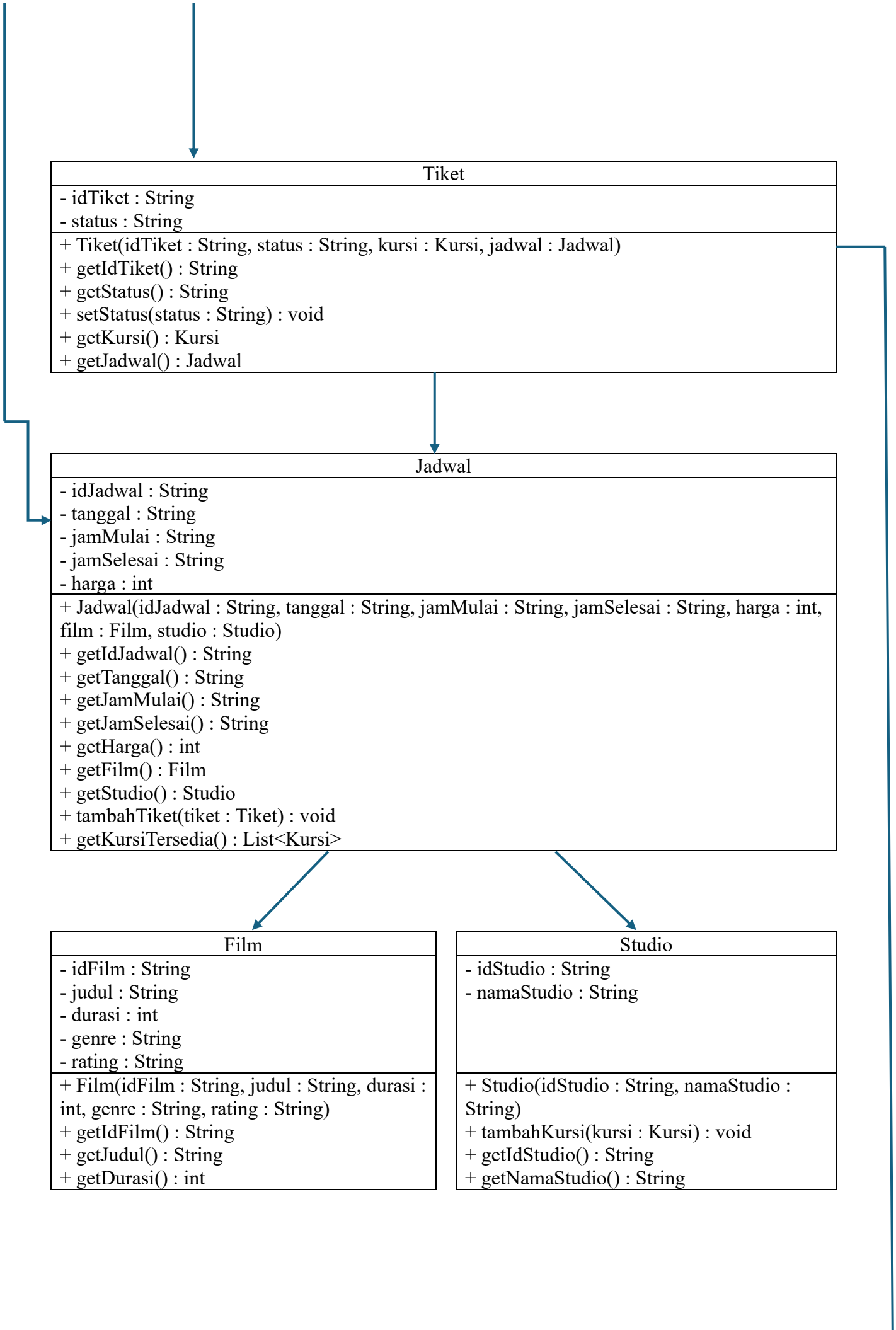
10. ntuk pembagian sutdi kasus, silahkan modulus 2 nomor terkahir NRP Anda dengan 6 (mod6), jika sisa bagi NRP Anda adalah 0 maka studikasus yang harus Anda kerjakan adalah studi kasus 6.

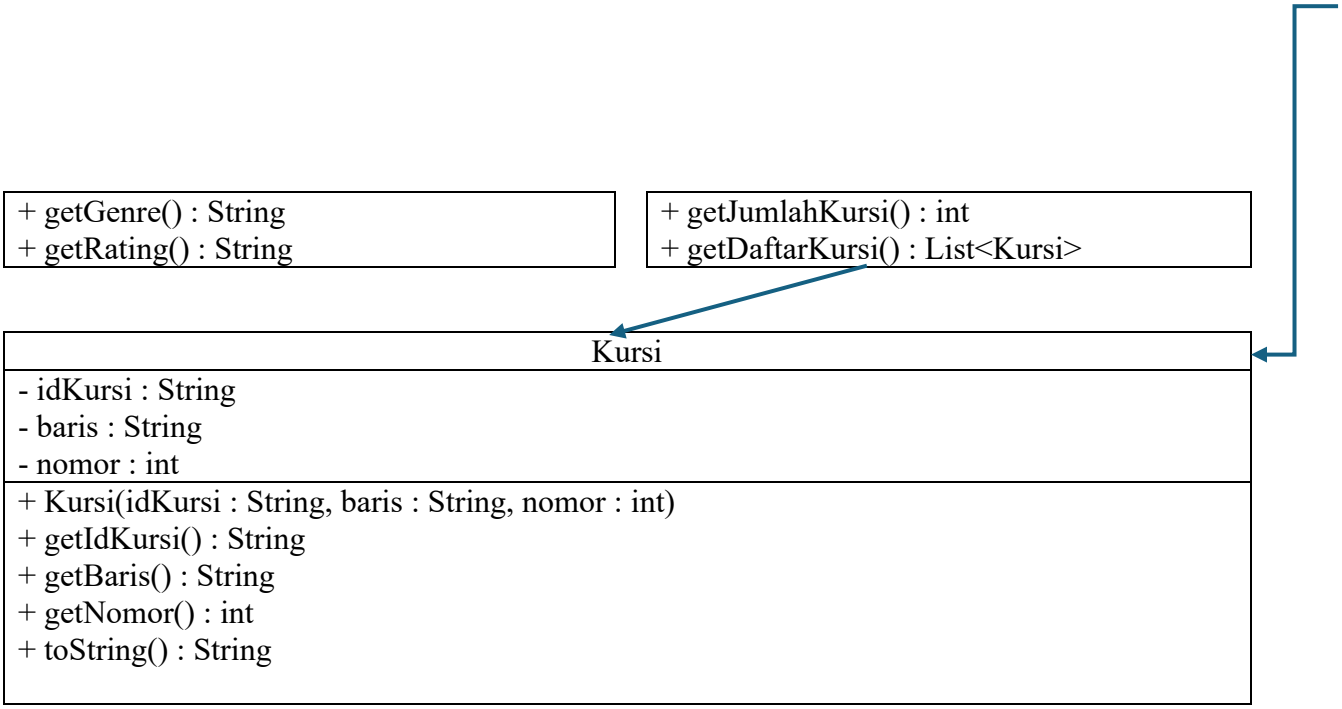
- 47 % 6 = 5 berarti Bioskop 212

UML DIAGRAM


Pengguna
- idPengguna : String
- namaLengkap : String
- noHP : String
- email : String
- alamat : String
- tanggalLahir : String
+ Pengguna(idPengguna : String, namaLengkap : String, noHP : String, email : String, alamat : String, tanggalLahir : String)
+ getIdPengguna() : String
+ getNamaLengkap() : String
+ getNoHP() : String
+ getEmail() : String
+ getAlamat() : String
+ getTanggalLahir() : String
+ setNamaLengkap(namaLengkap : String) : void
+ setNoHP(noHP : String) : void
+ setEmail(email : String) : void
+ setAlamat(alamat : String) : void
Pelanggan
+ Pelanggan(idPengguna : String, namaLengkap : String, noHP : String, email : String, alamat : String, tanggalLahir : String)
+ buatPesanan() : Pesanan
Kasir
- idKasir : String
+ Kasir(idKasir : String, idPengguna : String, namaLengkap : String, noHP : String, email : String, alamat : String, tanggalLahir : String)
+ getIdKasir() : String
+ prosesPesanan(pesanan : Pesanan) : void
Pesanan
- idPesanan : String
- tanggalPesan : String
- grandTotalHarga : int
- status : String
+ Pesanan(idPesanan : String, tanggalPesan : String, pelanggan : Pelanggan)
+ getIdPesanan() : String
+ getTanggalPesan() : String
+ getGrandTotalHarga() : int
+ getStatus() : String
+ getPelanggan() : Pelanggan
+ getKasir() : Kasir
Produk
- idProduk : String
- nama : String
- harga : int
- stok : int
+ Produk(idProduk : String, nama : String, harga : int, stok : int)
+ getIdProduk() : String
+ getNama() : String
+ getHarga() : int
+ getStok() : int
+ kurangiStok(jumlah : int) : void

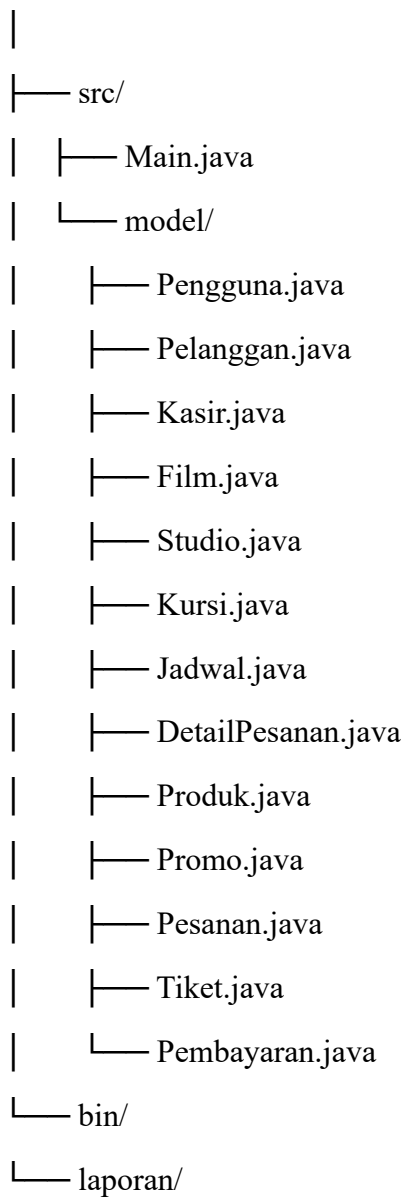

+ setKasir(kasir : Kasir) : void
+ getJadwal() : Jadwal
+ setJadwal(jadwal : Jadwal) : void
+ getPembayaran() : Pembayaran
+ setPembayaran(pembayaran : Pembayaran) : void
+ tambahKursi(kursi : Kursi) : void
+ getDaftarKursiDipesan() : List<Kursi>
+ tambahDetailPesanan(detail : DetailPesanan) : void
+ getDaftarDetail() : List<DetailPesanan>
+ hitungTotal() : int
+ batalPesanan() : void
Pembayaran
- idPembayaran : String
- metode : String
- totalBayar : int
- status : String
- tanggalBayar : String
+ Pembayaran(idPembayaran : String, metode : String, pesanan : Pesanan)
+ getIdPembayaran() : String
+ getMetode() : String
+ getTotalBayar() : int
+ getStatus() : String
+ getTanggalBayar() : String
+ getPesanan() : Pesanan
+ getPromo() : Promo
+ getDaftarTiket() : List<Tiket>
+ hitungTotalBayar(promo : Promo) : int
+ konfirmasiPembayaran() : void
DetailPesanan
- jumlah : int
- subTotal : int
+ DetailPesanan(produk : Produk, jumlah : int)
+ getJumlah() : int
+ getSubTotal() : int
+ getProduk() : Produk
+ hitungSubTotal() : int
Promo
- kodePromo : String
- potongan : int
- masaBerlaku : String
+ Promo(kodePromo : String, potongan : int, masaBerlaku : String)
+ getKodePromo() : String
+ getPotongan() : int
+ getMasaBerlaku() : String
+ isMasihBerlaku() : boolean


Tiket
- idTiket : String
- status : String
+ Tiket(idTiket : String, status : String, kursi : Kursi, jadwal : Jadwal)
+ getIdTiket() : String
+ getStatus() : String
+ setStatus(status : String) : void
+ getKursi() : Kursi
+ getJadwal() : Jadwal
Jadwal
- idJadwal : String
- tanggal : String
- jamMulai : String
- jamSelesai : String
- harga : int
+ Jadwal(idJadwal : String, tanggal : String, jamMulai : String, jamSelesai : String, harga : int, film : Film, studio : Studio)
+ getIdJadwal() : String
+ getTanggal() : String
+ getJamMulai() : String
+ getJamSelesai() : String
+ getHarga() : int
+ getFilm() : Film
+ getStudio() : Studio
+ tambahTiket(tiket : Tiket) : void
+ getKursiTersedia() : List<Kursi>
Film
- idFilm : String
- judul : String
- durasi : int
- genre : String
- rating : String
+ Film(idFilm : String, judul : String, durasi : int, genre : String, rating : String)
+ getIdFilm() : String
+ getJudul() : String
+ getDurasi() : int
Studio
- idStudio : String
- namaStudio : String
+ Studio(idStudio : String, namaStudio : String)
+ tambahKursi(kursi : Kursi) : void
+ getIdStudio() : String
+ getNamaStudio() : String


+ getGenre() : String
+ getRating() : String
+ getJumlahKursi() : int
+ getDaftarKursi() : List<Kursi>
Kursi
- idKursi : String
- baris : String
- nomor : int
+ Kursi(idKursi : String, baris : String, nomor : int)
+ getIdKursi() : String
+ getBaris() : String
+ getNomor() : int
+ toString() : String

Struktur Folder

uts/



Kode

Main.java

```
import model.*;

public class Main {
    public static void main(String[] args) {
        //film
        Film film1 = new Film("FLM-001", "The Last: Naruto the Movie", 112,
"Animation, Action, Romance", "13+");
        Film film2 = new Film("FLM-002", "Exhuma", 134, "Horror, Mystery,
Thriller", "17+");
        Film film3 = new Film("FLM-003", "Interstellar", 169, "Sci-Fi,
Adventure, Drama", "13+");

        System.out.println("[1] DAFTAR FILM YANG SEDANG TAYANG:");
        System.out.println("    1. " + film1.getJudul() + " | Durasi: " +
film1.getDurasi() + " mnt | Genre: " + film1.getGenre() + " | Rating: " +
film1.getRating());
        System.out.println("    2. " + film2.getJudul() + " | Durasi: " +
film2.getDurasi() + " mnt | Genre: " + film2.getGenre() + " | Rating: " +
film2.getRating());
        System.out.println("    3. " + film3.getJudul() + " | Durasi: " +
film3.getDurasi() + " mnt | Genre: " + film3.getGenre() + " | Rating: " +
film3.getRating());
        System.out.println();

        //stdio n kursi
        Studio studio1 = new Studio("STD-01", "Studio 1 Regular");
        Studio studio2 = new Studio("STD-02", "Studio 2 Premiere");

        for (int i = 0; i < 6; i++) {
            char hurufBaris = (char) ('A' + i);
            for (int no = 1; no <= 14; no++) {
                String idKursi = studio1.getIdStudio() + "-" + hurufBaris + no;
                studio1.tambahKursi(new Kursi(idKursi,
String.valueOf(hurufBaris), no));
            }
        }

        for (int i = 0; i < 4; i++) {
            char hurufBaris = (char) ('A' + i);
            for (int no = 1; no <= 8; no++) {
                String idKursi = studio2.getIdStudio() + "-" + hurufBaris + no;
```

```
                studio2.tambahKursi(new Kursi(idKursi,
String.valueOf(hurufBaris), no));
            }
        }

        System.out.println("[2] DAFTAR STUDIO & LAYOUT KURSI (Komposisi -
BR4):");
        System.out.println("    - " + studio1.getNamaStudio() + ": " +
studio1.getJumlahKursi() + " kursi (Baris A-F, No 1-14)");
        System.out.println("    - " + studio2.getNamaStudio() + ": " +
studio2.getJumlahKursi() + " kursi (Baris A-D, No 1-8)\n");

        //jadwal
        Jadwal jadwal1 = new Jadwal("JDW-01", "2026-10-20", "13:00", "15:00",
45000, film1, studio1);
        Jadwal jadwal2 = new Jadwal("JDW-02", "2026-10-20", "15:30", "17:45",
50000, film2, studio1);
        Jadwal jadwal3 = new Jadwal("JDW-03", "2026-10-21", "18:30", "21:20",
75000, film3, studio2);

        System.out.println("[3] JADWAL PENAYANGAN (Oktober 2026):");
        System.out.println("    1. [" + jadwal1.getIdJadwal() + "] " +
jadwal1.getFilm().getJudul()
                + " | " + jadwal1.getTanggal() + " " + jadwal1.getJamMulai() +
" di " + jadwal1.getStudio().getNamaStudio()
                + " | Rp " + jadwal1.getHarga());
        System.out.println("    2. [" + jadwal2.getIdJadwal() + "] " +
jadwal2.getFilm().getJudul()
                + " | " + jadwal2.getTanggal() + " " + jadwal2.getJamMulai() +
" di " + jadwal2.getStudio().getNamaStudio()
                + " | Rp " + jadwal2.getHarga());
        System.out.println("    3. [" + jadwal3.getIdJadwal() + "] " +
jadwal3.getFilm().getJudul()
                + " | " + jadwal3.getTanggal() + " " + jadwal3.getJamMulai() +
" di " + jadwal3.getStudio().getNamaStudio()
                + " | Rp " + jadwal3.getHarga());
        System.out.println();

        //produk n promo
        Produk caramelPopcorn = new Produk("PRD-01", "Caramel Popcorn (L)",
40000, 35);
        Produk saltyPopcorn = new Produk("PRD-02", "Salty Popcorn (M)", 30000,
25);
        Produk cocaCola = new Produk("PRD-03", "Coca-Cola (L)", 18000, 50);
        Produk pepsi = new Produk("PRD-04", "Pepsi Blue (L)", 18000, 40);
```

```
        Promo promoNobar = new Promo("NOBARSERU", 20000, "2026-10-31");

        System.out.println("[4] MENU MAKANAN / MINUMAN & PROMO:");
        System.out.println("    - " + caramelPopcorn.getNama() + " | Rp " +
caramelPopcorn.getHarga() + " (Stok: " + caramelPopcorn.getStok() + ")");
        System.out.println("    - " + saltyPopcorn.getNama() + " | Rp " +
saltyPopcorn.getHarga() + " (Stok: " + saltyPopcorn.getStok() + ")");
        System.out.println("    - " + cocaCola.getNama() + " | Rp " +
cocaCola.getHarga() + " (Stok: " + cocaCola.getStok() + ")");
        System.out.println("    - " + pepsi.getNama() + " | Rp " +
pepsi.getHarga() + " (Stok: " + pepsi.getStok() + ")");
        System.out.println("    - Kupon Promo: " + promoNobar.getKodePromo() +
" (Diskon: Rp " + promoNobar.getPotongan() + ", Exp: " +
promoNobar.getMasaBerlaku() + ")");
        System.out.println();

        //pengguna
        Pelanggan pelanggan1 = new Pelanggan("USR-01", "Bagus", "081234567890",
"", "", "");
        Kasir kasir1 = new Kasir("KSR-01", "USR-02", "Dewi Lestari",
"089876543210", "dewi@cinema.com", "Surabaya", "2000-08-20");

        System.out.println("[5] PENGGUNA TERLIBAT:");
        System.out.println("    Pelanggan : " + pelanggan1.getNamaLengkap());
        System.out.println("    Kasir     : " + kasir1.getNamaLengkap() + " [ID
Kasir: " + kasir1.getIdKasir() + "]\n");

        //skenario
        System.out.println("[6] DENAH KURSI SEBELUM PEMESANAN (Semua Kursi
Masih Kosong):");
        System.out.println("       ------------------ [ LAYAR BIOSKOP ] -------
-----------");
        String barisSaatIni1 = "";
        for (Kursi k : studio2.getDaftarKursi()) {
            if (!k.getBaris().equals(barisSaatIni1)) {
                if (!barisSaatIni1.isEmpty()) {
                    System.out.println();
                }
                barisSaatIni1 = k.getBaris();
                System.out.print("               ");
            }
            boolean kosong = false;
            for (Kursi ks : jadwal3.getKursiTersedia()) {
                if (ks.getIdKursi().equals(k.getIdKursi())) {
```

```
                    kosong = true;
                    break;
                }
            }
            if (kosong) {
                System.out.printf("[%2s] ", k.toString());
            } else {
                System.out.print("[XX] ");
            }
        }
        System.out.println("\n       ----------------------------------------
-------------");
        System.out.println("       Keterangan: [XX] = Terisi");

        System.out.println("\n[7] PELANGGAN MEMBUAT PESANAN:");
        Pesanan pesanan = pelanggan1.buatPesanan();
        pesanan.setJadwal(jadwal3);

        // Memesan kursi B3 dan B4 dengan helper yang ringkas
        for (Kursi k : studio2.getDaftarKursi()) {
            if (k.getBaris().equalsIgnoreCase("B") && (k.getNomor() == 3 ||
k.getNomor() == 4)) {
                pesanan.tambahKursi(k);
            }
        }

        // Menambah menu F&B: 1 Caramel Popcorn & 2 Pepsi Blue
        pesanan.tambahDetailPesanan(new DetailPesanan(caramelPopcorn, 1));
        pesanan.tambahDetailPesanan(new DetailPesanan(pepsi, 2));

        System.out.println("    ID Pesanan  : " + pesanan.getIdPesanan());
        System.out.println("    Film Dipilih: " + jadwal3.getFilm().getJudul()
+ " (" + studio2.getNamaStudio() + ")");
        System.out.println("    Kursi       : B3, B4 (2 tiket x Rp " +
jadwal3.getHarga() + ")");
        System.out.println("    F&B         : 1x " + caramelPopcorn.getNama() +
", 2x " + pepsi.getNama());
        System.out.println("    Grand Total : Rp " + pesanan.hitungTotal() +
"\n");

        // Kasir memproses pesanan
        kasir1.prosesPesanan(pesanan);
        System.out.println("    Status      : " + pesanan.getStatus() + "\n");

        //pembayaran
```

```
        System.out.println("[8] PROSES PEMBAYARAN & PENERBITAN TIKET:");
        Pembayaran pembayaran = new Pembayaran("BYR-2026-001", "QRIS",
pesanan);
        int totalSetelahDiskon = pembayaran.hitungTotalBayar(promoNobar);

        System.out.println("    Metode       : " + pembayaran.getMetode());
        System.out.println("    Kupon Promo  : " + promoNobar.getKodePromo() +
" (Potongan Rp " + promoNobar.getPotongan() + ")");
        System.out.println("    Total Bayar  : Rp " + totalSetelahDiskon +
"\n");

        // Konfirmasi pembayaran -> men-trigger penerbitan tiket dan pemotongan
stok F&B
        pembayaran.konfirmasiPembayaran();
        System.out.println("    Status Bayar : " + pembayaran.getStatus());
        System.out.println("    Tanggal Bayar: " +
pembayaran.getTanggalBayar());
        System.out.println("    Status Order : " + pesanan.getStatus());
        System.out.println("    Sisa Popcorn : " + caramelPopcorn.getStok() + "
(stok terpotong)");
        System.out.println("    Sisa Pepsi   : " + pepsi.getStok() + " (stok
terpotong)\n");

        System.out.println("[9] TIKET RESMI YANG DITERBITKAN:");
        for (Tiket t : pembayaran.getDaftarTiket()) {
            System.out.println("    [TIKET RESMI] ID: " + t.getIdTiket() +
                    " | Film: " + t.getJadwal().getFilm().getJudul() +
                    " | Studio: " + t.getJadwal().getStudio().getNamaStudio() +
                    " | Kursi: " + t.getKursi() +
                    " | Status: " + t.getStatus());
        }

        System.out.println("\n[10] DENAH KURSI SETELAH TIKET TERBIT (Kursi B3 &
B4 Bertanda [XX]):");
        System.out.println("       ------------------ [ LAYAR BIOSKOP ] -------
-----------");
        String barisSaatIni2 = "";
        for (Kursi k : studio2.getDaftarKursi()) {
            if (!k.getBaris().equals(barisSaatIni2)) {
                if (!barisSaatIni2.isEmpty()) {
                    System.out.println();
                }
                barisSaatIni2 = k.getBaris();
                System.out.print("               ");
            }
```

```
            boolean kosong = false;
            for (Kursi ks : jadwal3.getKursiTersedia()) {
                if (ks.getIdKursi().equals(k.getIdKursi())) {
                    kosong = true;
                    break;
                }
            }
            if (kosong) {
                System.out.printf("[%2s] ", k.toString());
            } else {
                System.out.print("[XX] ");
            }
        }
        System.out.println("\n       ----------------------------------------
-------------");
        System.out.println("       Keterangan: [XX] = Terisi");
    }
}
```

Film.java

```
package model;

public class Film {
    // atribut
    private String idFilm;
    private String judul;
    private int durasi;
    private String genre;
    private String rating;

    // constructor
    public Film(String idFilm, String judul, int durasi, String genre, String
rating) {
        this.idFilm = idFilm;
        this.judul = judul;
        this.durasi = durasi;
        this.genre = genre;
        this.rating = rating;
    }

    // getter
    public String getIdFilm() { return idFilm; }
    public String getJudul() { return judul; }
    public int getDurasi() { return durasi; }
    public String getGenre() { return genre; }
    public String getRating() { return rating; }
```

```
}
```

Studio.java

```java
package model;

public class Studio {
    // atribut
    private String idStudio;
    private String namaStudio;
    private Kursi[] daftarKursi;
    private int jumlahKursi;

    // constructor
    public Studio(String idStudio, String namaStudio) {
        this.idStudio = idStudio;
        this.namaStudio = namaStudio;
        this.daftarKursi = new Kursi[100];
        this.jumlahKursi = 0;
    }

    // method
    public void tambahKursi(Kursi kursi) {
        if (kursi != null) {
            if (jumlahKursi == daftarKursi.length) {
                Kursi[] temp = new Kursi[daftarKursi.length * 2];
                for (int i = 0; i < daftarKursi.length; i++) {
                    temp[i] = daftarKursi[i];
                }
                daftarKursi = temp;
            }
            daftarKursi[jumlahKursi++] = kursi;
        }
    }

    // getter
    public String getIdStudio() { return idStudio; }
    public String getNamaStudio() { return namaStudio; }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public Kursi[] getDaftarKursi() {
        Kursi[] hasil = new Kursi[jumlahKursi];
        for (int i = 0; i < jumlahKursi; i++) {
            hasil[i] = daftarKursi[i];
```

```
        }
        return hasil;
    }
}
```

Jadwal.java

```
package model;

public class Jadwal {
    // atribut
    private String idJadwal;
    private String tanggal;
    private String jamMulai;
    private String jamSelesai;
    private int harga;
    private Film film;
    private Studio studio;
    private Tiket[] daftarTiket;
    private int jumlahTiket;

    // constructor
    public Jadwal(String idJadwal, String tanggal, String jamMulai, String
jamSelesai,
                  int harga, Film film, Studio studio) {
        this.idJadwal = idJadwal;
        this.tanggal = tanggal;
        this.jamMulai = jamMulai;
        this.jamSelesai = jamSelesai;
        this.harga = harga;
        this.film = film;
        this.studio = studio;
        this.daftarTiket = new Tiket[100];
        this.jumlahTiket = 0;
    }

    // getter
    public String getIdJadwal() { return idJadwal; }
    public String getTanggal() { return tanggal; }
    public String getJamMulai() { return jamMulai; }
    public String getJamSelesai() { return jamSelesai; }
    public int getHarga() { return harga; }
    public Film getFilm() { return film; }
    public Studio getStudio() { return studio; }

    // method
    public void tambahTiket(Tiket tiket) {
```

```
        if (tiket == null) return;
        for (int i = 0; i < jumlahTiket; i++) {
            Tiket t = daftarTiket[i];
            if ("AKTIF".equalsIgnoreCase(t.getStatus()) &&
t.getKursi().equals(tiket.getKursi())) {
                throw new IllegalStateException("Kursi " + tiket.getKursi() + "
sudah memiliki tiket aktif pada jadwal ini (BR2)!");
            }
        }
        if (jumlahTiket == daftarTiket.length) {
            Tiket[] temp = new Tiket[daftarTiket.length * 2];
            for (int i = 0; i < daftarTiket.length; i++) {
                temp[i] = daftarTiket[i];
            }
            daftarTiket = temp;
        }
        daftarTiket[jumlahTiket++] = tiket;
    }

    public Kursi[] getKursiTersedia() {
        if (studio == null) return new Kursi[0];
        Kursi[] semuaKursi = studio.getDaftarKursi();

        boolean[] terisi = new boolean[semuaKursi.length];
        int countKosong = 0;

        for (int i = 0; i < semuaKursi.length; i++) {
            for (int j = 0; j < jumlahTiket; j++) {
                Tiket t = daftarTiket[j];
                if ("AKTIF".equalsIgnoreCase(t.getStatus()) &&
                    t.getKursi().getIdKursi().equals(semuaKursi[i].getIdKursi()
)) {
                    terisi[i] = true;
                    break;
                }
            }
            if (!terisi[i]) {
                countKosong++;
            }
        }

        Kursi[] hasil = new Kursi[countKosong];
        int index = 0;
        for (int i = 0; i < semuaKursi.length; i++) {
            if (!terisi[i]) {
```

```
                hasil[index++] = semuaKursi[i];
            }
        }
        return hasil;
    }
}
```

Tiket.java

```
package model;

public class Tiket {
    // atribut
    private String idTiket;
    private String status;
    private Kursi kursi;
    private Jadwal jadwal;

    // constructor
    public Tiket(String idTiket, String status, Kursi kursi, Jadwal jadwal) {
        this.idTiket = idTiket;
        this.status = status;
        this.kursi = kursi;
        this.jadwal = jadwal;
    }

    // getter n setter
    public String getIdTiket() { return idTiket; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public Kursi getKursi() { return kursi; }
    public Jadwal getJadwal() { return jadwal; }
}
```

Pelanggan.java

```
package model;

public class Pelanggan extends Pengguna {
    // constructor
    public Pelanggan(String idPengguna, String namaLengkap, String noHP,
                     String email, String alamat, String tanggalLahir) {
        super(idPengguna, namaLengkap, noHP, email, alamat, tanggalLahir);
    }

    // method
    public Pesanan buatPesanan() {
        String idPesanan = "PSN-" + System.currentTimeMillis();
        return new Pesanan(idPesanan, "2026-10-20", this);
```

```
    }
}
```

Pembayaran.java

```
package model;

public class Pembayaran {
    // atribut
    private String idPembayaran;
    private String metode;
    private int totalBayar;
    private String status;
    private String tanggalBayar;
    private Pesanan pesanan;
    private Promo promo;
    private Tiket[] daftarTiket;
    private int jumlahTiket;

    // constructor
    public Pembayaran(String idPembayaran, String metode, Pesanan pesanan) {
        if (pesanan == null) {
            throw new IllegalArgumentException("Pesanan tidak boleh null!");
        }
        this.idPembayaran = idPembayaran;
        this.metode = metode;
        this.pesanan = pesanan;
        this.status = "MENUNGGU";
        this.tanggalBayar = "-";
        this.totalBayar = pesanan.hitungTotal();
        this.daftarTiket = new Tiket[20];
        this.jumlahTiket = 0;
        this.pesanan.setPembayaran(this);
    }

    // getter n setter
    public String getIdPembayaran() { return idPembayaran; }
    public String getMetode() { return metode; }
    public int getTotalBayar() { return totalBayar; }
    public String getStatus() { return status; }
    public String getTanggalBayar() { return tanggalBayar; }
    public Pesanan getPesanan() { return pesanan; }
    public Promo getPromo() { return promo; }

    public Tiket[] getDaftarTiket() {
        Tiket[] hasil = new Tiket[jumlahTiket];
        for (int i = 0; i < jumlahTiket; i++) {
```

```
            hasil[i] = daftarTiket[i];
        }
        return hasil;
    }

    public void setStatusInternal(String status) { this.status = status; }

    // method
    public int hitungTotalBayar(Promo promo) {
        int baseTotal = pesanan.hitungTotal();
        this.promo = promo;
        if (promo != null && promo.isMasihBerlaku()) {
            this.totalBayar = Math.max(0, baseTotal - promo.getPotongan());
        } else {
            this.totalBayar = baseTotal;
        }
        return this.totalBayar;
    }

    public void konfirmasiPembayaran() {
        if ("LUNAS".equalsIgnoreCase(this.status)) {
            System.out.println("Pembayaran sudah lunas sebelumnya.");
            return;
        }
        this.status = "LUNAS";
        this.tanggalBayar = "2026-10-20";
        this.pesanan.setStatusInternal("SELESAI");

        for (DetailPesanan detail : pesanan.getDaftarDetail()) {
            detail.getProduk().kurangiStok(detail.getJumlah());
        }

        Jadwal jadwal = pesanan.getJadwal();
        if (jadwal != null) {
            int noTiket = 1;
            for (Kursi kursi : pesanan.getDaftarKursiDipesan()) {
                String idTiket = "TKT-" + pesanan.getIdPesanan() + "-" +
(noTiket++);
                Tiket tiket = new Tiket(idTiket, "AKTIF", kursi, jadwal);
                jadwal.tambahTiket(tiket);

                if (jumlahTiket == daftarTiket.length) {
                    Tiket[] temp = new Tiket[daftarTiket.length * 2];
                    for (int i = 0; i < daftarTiket.length; i++) {
                        temp[i] = daftarTiket[i];
```

```
                    }
                    daftarTiket = temp;
                }
                daftarTiket[jumlahTiket++] = tiket;
            }
        }
        System.out.println("Pembayaran berhasil dikonfirmasi! " + jumlahTiket +
" tiket telah diterbitkan.");
    }
}
```

Produk.java

```
package model;

public class Produk {
    // atribut
    private String idProduk;
    private String nama;
    private int harga;
    private int stok;

    // constructor
    public Produk(String idProduk, String nama, int harga, int stok) {
        this.idProduk = idProduk;
        this.nama = nama;
        this.harga = harga;
        this.stok = stok;
    }

    // getter
    public String getIdProduk() { return idProduk; }
    public String getNama() { return nama; }
    public int getHarga() { return harga; }
    public int getStok() { return stok; }

    // method
    public void kurangiStok(int jumlah) {
        if (jumlah <= 0) {
            throw new IllegalArgumentException("Jumlah harus lebih dari 0");
        }
        if (this.stok < jumlah) {
            throw new IllegalStateException("Stok produk tidak mencukupi. Sisa
stok: " + this.stok);
        }
        this.stok -= jumlah;
    }
```

```
}
```

Pesanan.java

```
package model;

public class Pesanan {
    // atribut
    private String idPesanan;
    private String tanggalPesan;
    private int grandTotalHarga;
    private String status;

    // relasi
    private Pelanggan pelanggan;
    private Kasir kasir;
    private Jadwal jadwal;
    private Kursi[] daftarKursiDipesan;
    private int jumlahKursiDipesan;
    private DetailPesanan[] daftarDetail;
    private int jumlahDetail;
    private Pembayaran pembayaran;

    // constructor
    public Pesanan(String idPesanan, String tanggalPesan, Pelanggan pelanggan)
{
        this.idPesanan = idPesanan;
        this.tanggalPesan = tanggalPesan;
        this.pelanggan = pelanggan;
        this.status = "MENUNGGU";
        this.grandTotalHarga = 0;
        this.daftarKursiDipesan = new Kursi[20];
        this.jumlahKursiDipesan = 0;
        this.daftarDetail = new DetailPesanan[20];
        this.jumlahDetail = 0;
    }

    // getter n setter
    public String getIdPesanan() { return idPesanan; }
    public String getTanggalPesan() { return tanggalPesan; }
    public int getGrandTotalHarga() { return grandTotalHarga; }
    public String getStatus() { return status; }
    public Pelanggan getPelanggan() { return pelanggan; }
    public Kasir getKasir() { return kasir; }
    public void setKasir(Kasir kasir) { this.kasir = kasir; }
    public Jadwal getJadwal() { return jadwal; }
    public void setJadwal(Jadwal jadwal) { this.jadwal = jadwal; }
```

```
    public Pembayaran getPembayaran() { return pembayaran; }
    public void setPembayaran(Pembayaran pembayaran) { this.pembayaran =
pembayaran; }
    public void setStatusInternal(String status) { this.status = status; }

    public Kursi[] getDaftarKursiDipesan() {
        Kursi[] hasil = new Kursi[jumlahKursiDipesan];
        for (int i = 0; i < jumlahKursiDipesan; i++) {
            hasil[i] = daftarKursiDipesan[i];
        }
        return hasil;
    }

    public DetailPesanan[] getDaftarDetail() {
        DetailPesanan[] hasil = new DetailPesanan[jumlahDetail];
        for (int i = 0; i < jumlahDetail; i++) {
            hasil[i] = daftarDetail[i];
        }
        return hasil;
    }

    // method
    public void tambahKursi(Kursi kursi) {
        if (kursi != null) {
            for (int i = 0; i < jumlahKursiDipesan; i++) {
                if (daftarKursiDipesan[i].equals(kursi)) {
                    return;
                }
            }
            if (jumlahKursiDipesan == daftarKursiDipesan.length) {
                Kursi[] temp = new Kursi[daftarKursiDipesan.length * 2];
                for (int i = 0; i < daftarKursiDipesan.length; i++) {
                    temp[i] = daftarKursiDipesan[i];
                }
                daftarKursiDipesan = temp;
            }
            daftarKursiDipesan[jumlahKursiDipesan++] = kursi;
        }
    }

    public void tambahDetailPesanan(DetailPesanan detail) {
        if (detail != null) {
            if (jumlahDetail == daftarDetail.length) {
                DetailPesanan[] temp = new DetailPesanan[daftarDetail.length *
2];
```

```
                for (int i = 0; i < daftarDetail.length; i++) {
                    temp[i] = daftarDetail[i];
                }
                daftarDetail = temp;
            }
            daftarDetail[jumlahDetail++] = detail;
        }
    }

    public int hitungTotal() {
        int total = 0;
        for (int i = 0; i < jumlahDetail; i++) {
            total += daftarDetail[i].hitungSubTotal();
        }
        if (jadwal != null && jumlahKursiDipesan > 0) {
            total += jumlahKursiDipesan * jadwal.getHarga();
        }
        this.grandTotalHarga = total;
        return this.grandTotalHarga;
    }

    public void batalPesanan() {
        if ("SELESAI".equalsIgnoreCase(this.status)) {
            throw new IllegalStateException("Pesanan yang sudah selesai tidak
dapat dibatalkan!");
        }
        this.status = "BATAL";
        if (this.pembayaran != null &&
!"LUNAS".equalsIgnoreCase(this.pembayaran.getStatus())) {
            this.pembayaran.setStatusInternal("BATAL");
        }
        System.out.println("Pesanan " + idPesanan + " berhasil dibatalkan.");
    }
}
```

Kasir.java

```
package model;

public class Kasir extends Pengguna {
    // atribut
    private String idKasir;

    // constructor
    public Kasir(String idKasir, String idPengguna, String namaLengkap,
                 String noHP, String email, String alamat, String tanggalLahir)
{
```

```
        super(idPengguna, namaLengkap, noHP, email, alamat, tanggalLahir);
        this.idKasir = idKasir;
    }

    // getter
    public String getIdKasir() { return idKasir; }

    // method
    public void prosesPesanan(Pesanan pesanan) {
        if (pesanan != null && "MENUNGGU".equals(pesanan.getStatus())) {
            pesanan.setStatusInternal("DIPROSES");
            System.out.println("Kasir " + getNamaLengkap() + " memproses
pesanan: " + pesanan.getIdPesanan());
        }
    }
}
```

Promo.java

```
package model;

public class Produk {
    // atribut
    private String idProduk;
    private String nama;
    private int harga;
    private int stok;

    // constructor
    public Produk(String idProduk, String nama, int harga, int stok) {
        this.idProduk = idProduk;
        this.nama = nama;
        this.harga = harga;
        this.stok = stok;
    }

    // getter
    public String getIdProduk() { return idProduk; }
    public String getNama() { return nama; }
    public int getHarga() { return harga; }
    public int getStok() { return stok; }

    // method
    public void kurangiStok(int jumlah) {
        if (jumlah <= 0) {
            throw new IllegalArgumentException("Jumlah harus lebih dari 0");
        }
```

```
        if (this.stok < jumlah) {
            throw new IllegalStateException("Stok produk tidak mencukupi. Sisa
stok: " + this.stok);
        }
        this.stok -= jumlah;
    }
}
```

DetailPesanan.java

```
package model;

public class DetailPesanan {
    // atribut
    private int jumlah;
    private int subTotal;
    private Produk produk;

    // constructor
    public DetailPesanan(Produk produk, int jumlah) {
        if (produk == null) {
            throw new IllegalArgumentException("Produk tidak boleh null");
        }
        if (jumlah <= 0) {
            throw new IllegalArgumentException("Jumlah harus lebih dari 0");
        }
        this.produk = produk;
        this.jumlah = jumlah;
        this.subTotal = hitungSubTotal();
    }

    // method
    public int hitungSubTotal() {
        if (produk != null) {
            this.subTotal = this.jumlah * produk.getHarga();
        }
        return this.subTotal;
    }

    // getter
    public int getJumlah() { return jumlah; }
    public int getSubTotal() { return subTotal; }
    public Produk getProduk() { return produk; }
}
```

Kursi.java

```
package model;
```

```java
public class Kursi {
    // atribut
    private String idKursi;
    private String baris;
    private int nomor;

    // constructor
    public Kursi(String idKursi, String baris, int nomor) {
        this.idKursi = idKursi;
        this.baris = baris;
        this.nomor = nomor;
    }

    // getter
    public String getIdKursi() { return idKursi; }
    public String getBaris() { return baris; }
    public int getNomor() { return nomor; }

    @Override
    public String toString() {
        return baris + nomor;
    }
}
```

Output

```
[1] DAFTAR FILM YANG SEDANG TAYANG:
    1. The Last: Naruto the Movie | Durasi: 112 mnt | Genre: Animation, Action, Romance | Rating: 13+
    2. Exhuma | Durasi: 134 mnt | Genre: Horror, Mystery, Thriller | Rating: 17+
    3. Interstellar | Durasi: 169 mnt | Genre: Sci-Fi, Adventure, Drama | Rating: 13+

[2] DAFTAR STUDIO & LAYOUT KURSI (Komposisi - BR4):
    - Studio 1 Regular: 84 kursi (Baris A-F, No 1-14)
    - Studio 2 Premiere: 32 kursi (Baris A-D, No 1-8)

[3] JADWAL PENAYANGAN (Oktober 2026):
    1. [JDW-01] The Last: Naruto the Movie | 2026-10-20 13:00 di Studio 1 Regular | Rp 45000
    2. [JDW-02] Exhuma | 2026-10-20 15:30 di Studio 1 Regular | Rp 50000
    3. [JDW-03] Interstellar | 2026-10-21 18:30 di Studio 2 Premiere | Rp 75000

[4] MENU MAKANAN / MINUMAN & PROMO:
    - Caramel Popcorn (L) | Rp 40000 (Stok: 35)
    - Salty Popcorn (M) | Rp 30000 (Stok: 25)
    - Coca-Cola (L) | Rp 18000 (Stok: 50)
    - Pepsi Blue (L) | Rp 18000 (Stok: 40)
    - Kupon Promo: NOBARSERU (Diskon: Rp 20000, Exp: 2026-10-31)

[5] PENGGUNA TERLIBAT:
    Pelanggan : Bagus
    Kasir     : Dewi Lestari [ID Kasir: KSR-01]
```

```
[5] PENGGUNA TERLIBAT:
    Pelanggan : Bagus
    Kasir     : Dewi Lestari [ID Kasir: KSR-01]

[6] DENAH KURSI SEBELUM PEMESANAN (Semua Kursi Masih Kosong):
       ------------------ [ LAYAR BIOSKOP ] ------------------
              [A1] [A2] [A3] [A4] [A5] [A6] [A7] [A8]
              [B1] [B2] [B3] [B4] [B5] [B6] [B7] [B8]
              [C1] [C2] [C3] [C4] [C5] [C6] [C7] [C8]
              [D1] [D2] [D3] [D4] [D5] [D6] [D7] [D8]
       ----------------------------------------------------------
       Keterangan: [XX] = Terisi

[7] PELANGGAN MEMBUAT PESANAN:
    ID Pesanan  : PSN-1792352224286
    Film Dipilih: Interstellar (Studio 2 Premiere)
    Kursi       : B3, B4 (2 tiket x Rp 75000)
    F&B         : 1x Caramel Popcorn (L), 2x Pepsi Blue (L)
    Grand Total : Rp 226000

Kasir Dewi Lestari memproses pesanan: PSN-1792352224286
    Status      : DIPROSES

[8] PROSES PEMBAYARAN & PENERBITAN TIKET:
    Metode       : QRIS
    Kupon Promo  : NOBARSERU (Potongan Rp 20000)
    Total Bayar  : Rp 206000

Pembayaran berhasil dikonfirmasi! 2 tiket telah diterbitkan.
    Status Bayar : LUNAS
    Tanggal Bayar: 2026-10-20
    Status Order : SELESAI
    Sisa Popcorn : 34 (stok terpotong)
    Sisa Pepsi   : 38 (stok terpotong)
```

```
[9] TIKET RESMI YANG DITERBITKAN:
    [TIKET RESMI] ID: TKT-PSN-1792352224286-1 | Film: Interstellar | Studio: Studio 2 Premiere | Kursi: B3 | Status: AKTIF
    [TIKET RESMI] ID: TKT-PSN-1792352224286-2 | Film: Interstellar | Studio: Studio 2 Premiere | Kursi: B4 | Status: AKTIF

[10] DENAH KURSI SETELAH TIKET TERBIT (Kursi B3 & B4 Bertanda [XX]):
       ----------------- [ LAYAR BIOSKOP ] -----------------
              [A1] [A2] [A3] [A4] [A5] [A6] [A7] [A8]
              [B1] [B2] [XX] [XX] [B5] [B6] [B7] [B8]
              [C1] [C2] [C3] [C4] [C5] [C6] [C7] [C8]
              [D1] [D2] [D3] [D4] [D5] [D6] [D7] [D8]
       ------------------------------------------------------
       Keterangan: [XX] = Terisi
```

Kesimpulan

Program sudah menerapkan prinsip OOP pada studi kasus sistem pemesanan tiket bioskop melalui 13 class Java yang saling berinteraksi secara modular. Enkapsulasi diterapkan melalui atribut private yang diakses melalui getter/setter serta dilengkapi validasi logika bisnis, seperti validasi stok pada kurangiStok() dan validasi status pada batalPesanan(). Inheritance dan Abstraksi diterapkan lewat class abstract Pengguna yang diturunkan ke subclass Pelanggan dan Kasir menggunakan super constructor. Relasi Komposisi diterapkan secara nyata pada Studio terhadap entitas Kursi. Array of objects diterapkan secara murni tanpa collection framework untuk menyimpan dan mengolah kumpulan data, seperti Kursi[] pada Studio, Tiket[] pada Jadwal dan Pembayaran, serta Kursi[] dan DetailPesanan[] pada Pesanan. Program juga telah berhasil menjalankan seluruh alur bisnis utama—mulai dari pengecekan ketersediaan kursi pada denah studio (BR1 & BR3), pemesanan tiket dan produk F&B, pemrosesan pesanan oleh kasir, penerapan diskon promo, konfirmasi pembayaran, penerbitan tiket otomatis, hingga pencegahan double-booking (BR2)—serta berhasil dikompilasi dan dijalankan secara terstruktur melalui main().